

Apuntes de Compiladores e Intérpretes

Kevin Jiménez Escalante (2024253395)
01 de octubre de 2026

NFA- ϵ : Transiciones- ϵ

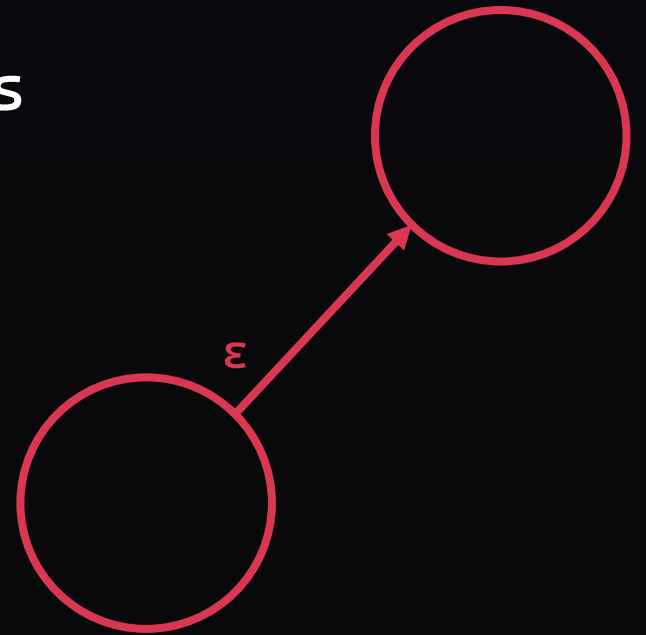
Propiedades:

Si hay transición ϵ , se puede saltar entre estados sin consumir símbolos.

No es obligatorio usarlas.

Tienen naturaleza no determinística.

Todos los estados tienen una transición ϵ (implícita) a sí mismos.



Definición de NFA- ϵ

Una NFA- ϵ se define con el quinteto $(Q, \Sigma, \delta, q_0, F)$ donde:

- Q es un conjunto de estados finitos
- Σ es un alfabeto
- δ es la función de transición $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$
- q_0 es el estado inicial
- $F \subseteq Q$ es el conjunto de estados de aceptación

¿No suena familiar?

Definición de NFA- ϵ

Esta definición es muy similar a las definiciones de DFA y NFA, la única diferencia entre estas es la función de transición δ .

- En DFA $\delta: Q \times \Sigma \rightarrow Q$
- En NFA $\delta: Q \times \Sigma \rightarrow \mathcal{P}(Q)$
- En NFA- ϵ $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$

¿Y todos estos símbolos?

No hace falta explicar que es $\mathcal{P}(Q)$ porque todo mundo lo vio en discreta y prestó atención en la clase :D

Pero como recordatorio, $\mathcal{P}(Q)$ es el conjunto que tiene todos los posibles subconjuntos de Q .



4 elementos

$P(Q)$



2^4 elementos

$$\text{DFA } \delta: Q \times \Sigma \rightarrow Q$$

Recibe un elemento de Q y otro de Σ , devuelve un elemento de Q

$$\text{NFA } \delta: Q \times \Sigma \rightarrow \mathcal{P}(Q)$$

Recibe un elemento de Q y otro de Σ , devuelve un subconjunto de Q

$$\text{NFA-}\varepsilon \delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$$

Recibe un elemento de Q y otro de Σ (o también ε), devuelve un subconjunto de Q

Teorema ultra chiva:

Sea M un NFA- $\varepsilon = (Q, \Sigma, \delta, q_0, F)$.

Existe un NFA $M' = (\mathcal{P}(Q), \Sigma, \delta', q_0, F')$

Que reconoce exactamente el mismo lenguaje

Recordemos que ya sabemos convertir una NFA en una DFA...

Lo que significa que podemos convertir una NFA- ε en una DFA...

¡Osea que las tres máquinas son igual de poderosas!



Pero, ¿¿¿cómo convertimos una en otra???

Se usa el cierre ϵ !



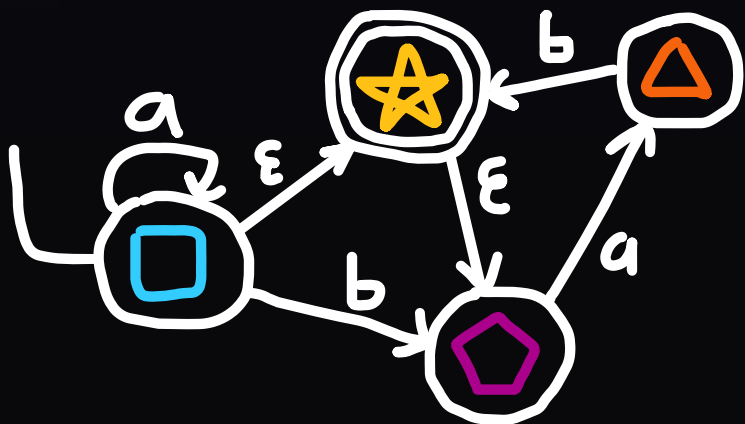
Cierre ε

El cierre (o alcance) ε es una función que recibe un estado de Q y regresa el subconjunto de Q con todos los estados que se pueden alcanzar mediante transiciones ε .

Implícitamente siempre hay una transición a sí mismo por lo que nunca es vacío.

Para todo cálculo hay que usar $\text{Cierre-}\varepsilon(p)$ en lugar de p , incluso para calcular $\text{Cierre-}\varepsilon(p)$:D

Cierre ϵ



Estado



Cierre- ϵ

{  

{ 

{

{

Convertir NFA- ϵ a DFA

Entonces, queremos convertir una NFA- ϵ en una DFA, recordemos un par de cosas:

NFA- ϵ original: $M = (Q, \Sigma, \delta, q_0, F)$

DFA objetivo: $M' = (\mathcal{P}(Q), \Sigma, \delta', q_0, F')$

Antes de ir al algoritmo diabólico hay que tomar en cuenta un par de cosas para convertir una NFA- ϵ en una DFA:

- Se usa el mismo alfabeto Σ
- Los estados de M' son subconjuntos de los estados Q de M
- Hay que calcular una nueva función de transición δ'
- F' es un conjunto de subconjuntos de Q

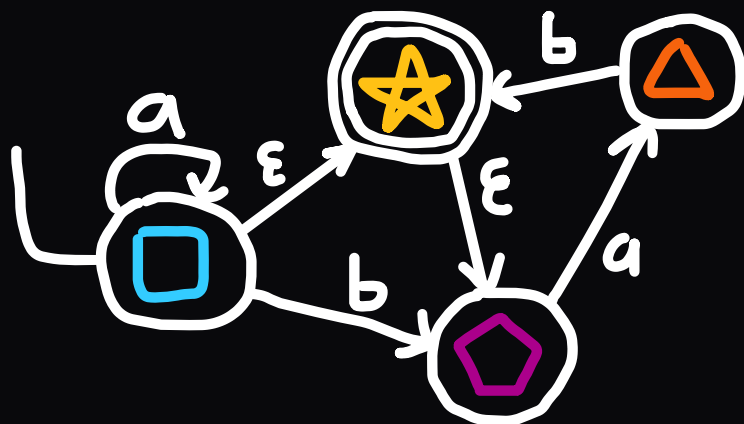
El algoritmo diabólico I

Crear una tabla con sólo una fila (por ahora) y una columna por símbolo.

	a	b
{□★◇}		

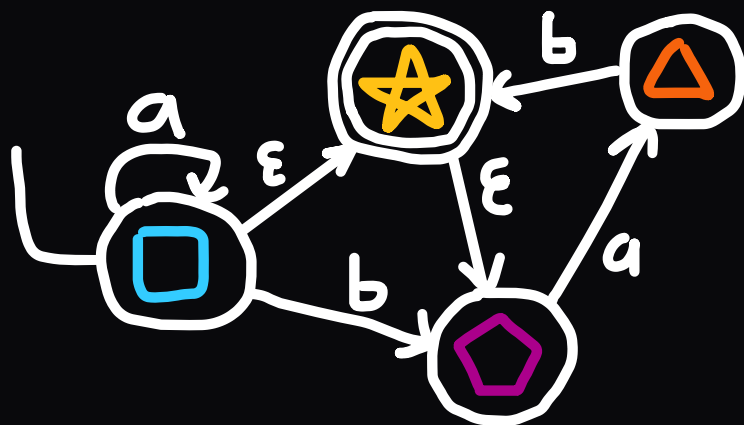
En la primera fila poner el cierre- ϵ del estado inicial.

Estado inicial: □
Cierre- ϵ de □: {□★◇}



El algoritmo diabólico II

Por cada elemento del cierre- ϵ , poner el cierre- ϵ del estado al que va a parar si recibe un símbolo.



	a	b
$\{\square \star \diamond\}$	$\{\square \star \diamond \triangle\}$	

Si $\square$ recibe a $\rightarrow \square$
Cierre- $\epsilon(\square) = \{\square \star \diamond\}$

Si $\star$ recibe a $\rightarrow \emptyset$

Si $\diamond$ recibe a $\rightarrow \triangle$
Cierre- $\epsilon(\triangle) = \{\triangle\}$

El algoritmo diabólico II

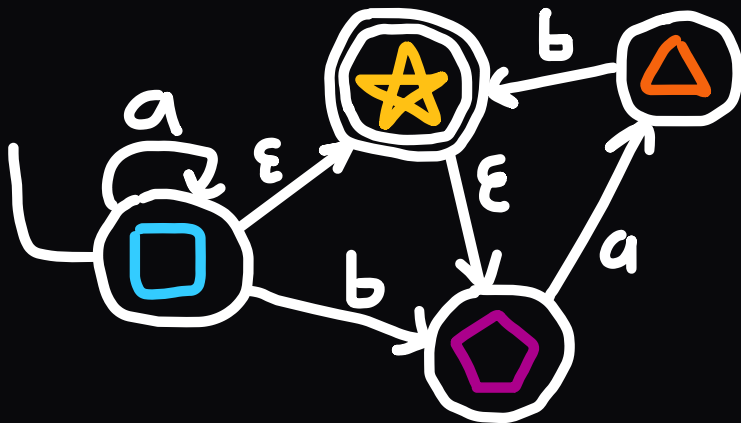
Por cada elemento del cierre- ϵ , poner el cierre- ϵ del estado al que va a parar si recibe un símbolo.

	a	b
$\{\square \star \diamondsuit\}$	$\{\square \star \diamondsuit \triangle\}$	$\{\diamondsuit\}$

Si $\square$ recibe b $\rightarrow \diamondsuit$
Cierre- $\epsilon(\diamondsuit) = \{\diamondsuit\}$

Si $\star$ recibe b $\rightarrow \emptyset$

Si $\diamondsuit$ recibe b $\rightarrow \emptyset$



El algoritmo diabólico III

Si hay un conjunto que no tenga su “entrada” en la tabla, repetir hasta que no queden “nuevos”.

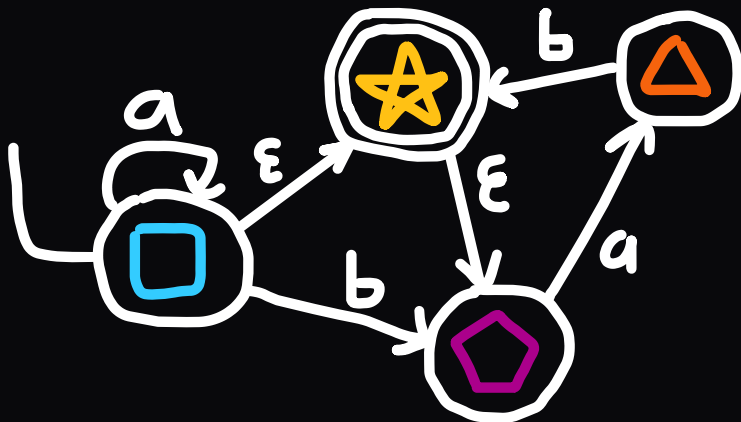
	a	b
$\{\square \star \diamond\}$	$\{\square \star \diamond \triangle\}$	$\{\diamond\}$
$\{\square \star \diamond \triangle\}$	$\{\square \star \diamond \triangle\}$	

Si $\square$ recibe a $\rightarrow \square$
Cierre- $\epsilon(\square) = \{\square \star \diamond\}$

Si $\star$ recibe a $\rightarrow \emptyset$

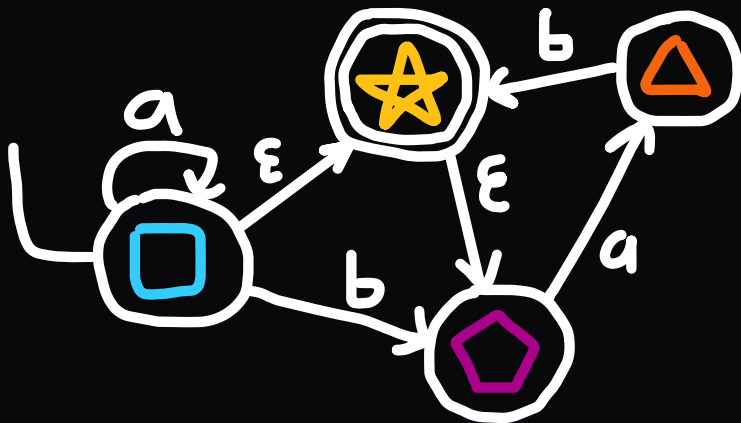
Si $\diamond$ recibe a $\rightarrow \triangle$
Cierre- $\epsilon(\triangle) = \{\triangle\}$

Si $\triangle$ recibe a $\rightarrow \emptyset$



El algoritmo diabólico III

Si hay un conjunto que no tenga su “entrada” en la tabla, repetir hasta que no queden “nuevos”.



	a	b
$\{\square \star \diamondsuit\}$	$\{\square \star \diamondsuit \triangle\}$	$\{\diamondsuit\}$
$\{\square \star \diamondsuit \triangle\}$	$\{\square \star \diamondsuit \triangle\}$	$\{\star \diamondsuit\}$

Si $\square$ recibe $b \rightarrow \diamondsuit$

Cierre- $\epsilon(\diamondsuit) = \{\diamondsuit\}$

Si $\star$ recibe $b \rightarrow \emptyset$

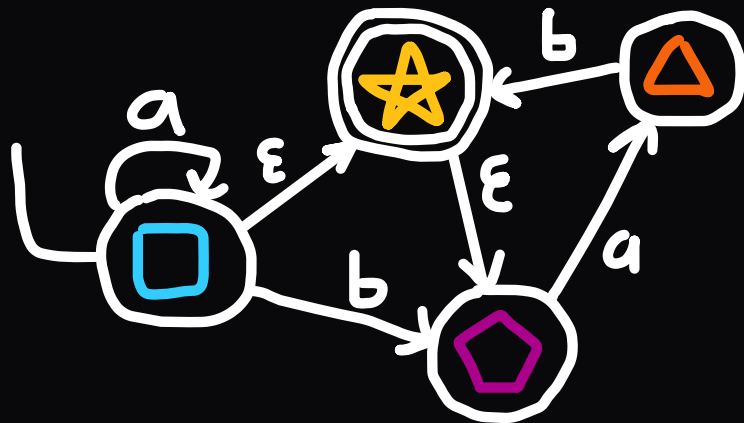
Si $\diamondsuit$ recibe $b \rightarrow \emptyset$

Si $\triangle$ recibe $b \rightarrow \star$

Cierre- $\epsilon(\diamondsuit) = \{\star \diamondsuit\}$

El algoritmo diabólico III

Si hay un conjunto que no tenga su “entrada” en la tabla, repetir hasta que no queden “nuevos”.



	a	b
$\{\square \star \diamondsuit\}$	$\{\square \star \diamondsuit \triangle\}$	$\{\diamondsuit\}$
$\{\square \star \diamondsuit \triangle\}$	$\{\square \star \diamondsuit \triangle\}$	$\{\star \diamondsuit\}$
$\{\diamondsuit\}$	$\{\triangle\}$	$\emptyset$

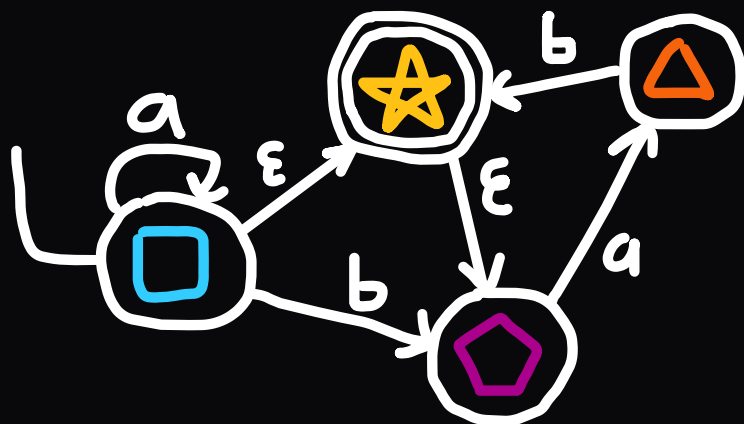
Si $\diamondsuit$ recibe a $\rightarrow \triangle$
Cierre- $\epsilon(\triangle) = \{\triangle\}$

Si $\diamondsuit$ recibe b $\rightarrow \emptyset$

El algoritmo diabólico III

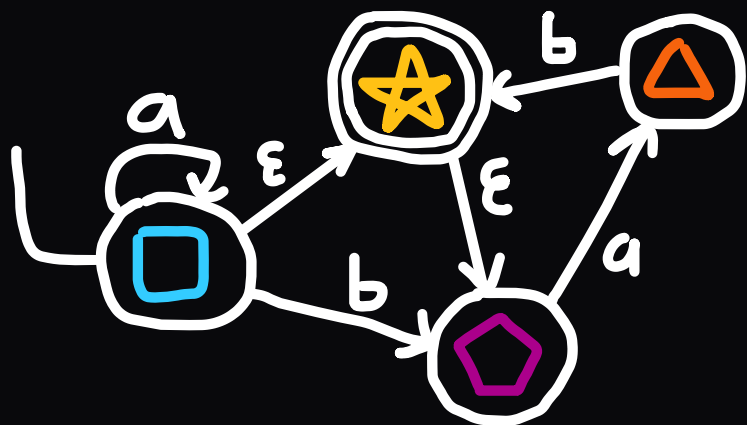
Si hay un conjunto que no tenga su “entrada” en la tabla, repetir hasta que no queden “nuevos”.

	a	b
$\{\square \star \diamond\}$	$\{\square \star \diamond \triangle\}$	$\{\diamond\}$
$\{\square \star \diamond \triangle\}$	$\{\square \star \diamond \triangle\}$	$\{\star \diamond\}$
$\{\diamond\}$	$\{\triangle\}$	$\emptyset$
$\{\star \diamond\}$	$\{\triangle\}$	$\emptyset$



El algoritmo diabólico III

Si hay un conjunto que no tenga su “entrada” en la tabla, repetir hasta que no queden “nuevos”.

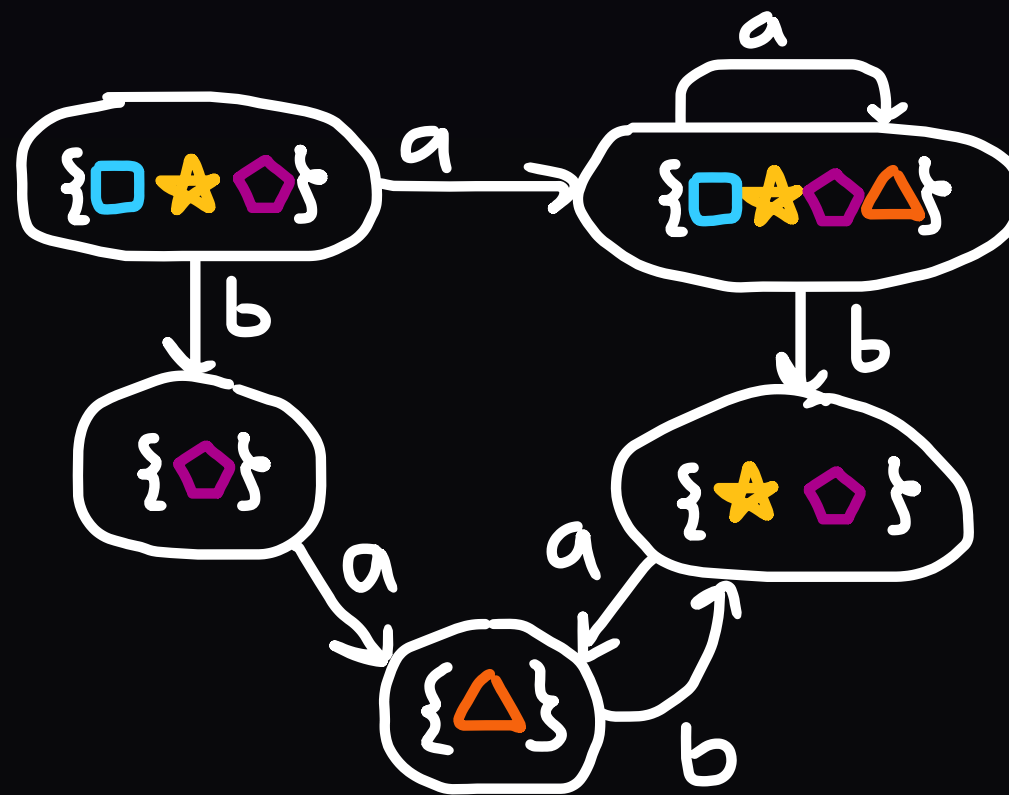


	a	b
{□★⬠}	{□★⬠△}	{⬠}
{□★⬠△}	{□★⬠△}	{★⬠}
{⬠}	{△}	∅
{★⬠}	{△}	∅
{△}	∅	{★⬠}

El algoritmo diabólico IV

Cuando no queden conjuntos sin entrada, hemos terminado la tabla. Esta tabla se puede usar para crear una máquina determinística :D

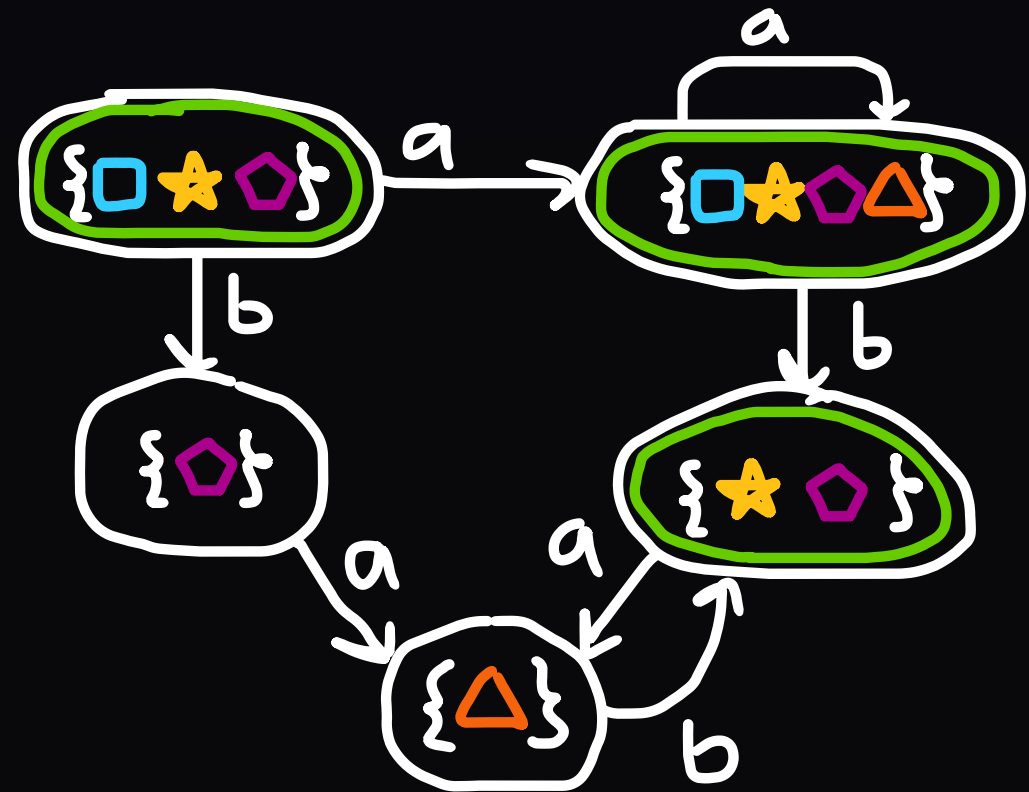
	a	b
$\{\square, \star, \diamond\}$	$\{\square, \star, \diamond, \triangle\}$	$\{\diamond\}$
$\{\square, \star, \diamond, \triangle\}$	$\{\square, \star, \diamond, \triangle\}$	$\{\star, \diamond\}$
$\{\diamond\}$	$\{\triangle\}$	$\emptyset$
$\{\star, \diamond\}$	$\{\triangle\}$	$\emptyset$
$\{\triangle\}$	$\emptyset$	$\{\star, \diamond\}$



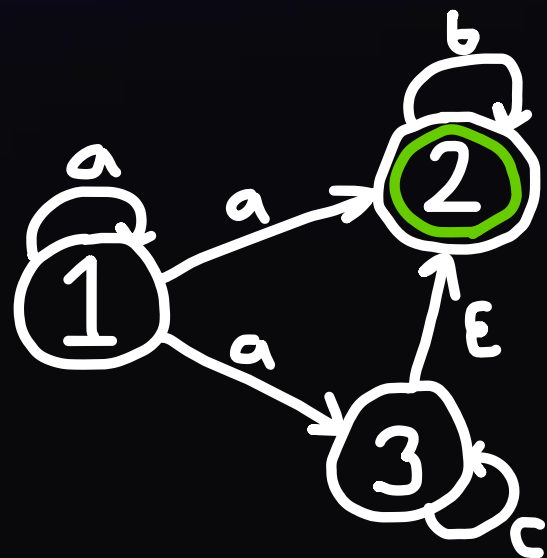
El algoritmo diabólico V

Recordemos que el estado de aceptación es $\star$, marcar cada estado que lo contenga como de aceptación.

	a	b
$\{\square \star \diamond\}$	$\{\square \star \diamond \triangle\}$	$\{\diamond\}$
$\{\square \star \diamond \triangle\}$	$\{\square \star \diamond \triangle\}$	$\{\star \diamond\}$
$\{\diamond\}$	$\{\triangle\}$	$\emptyset$
$\{\star \diamond\}$	$\{\triangle\}$	$\emptyset$
$\{\triangle\}$	$\emptyset$	$\{\star \diamond\}$

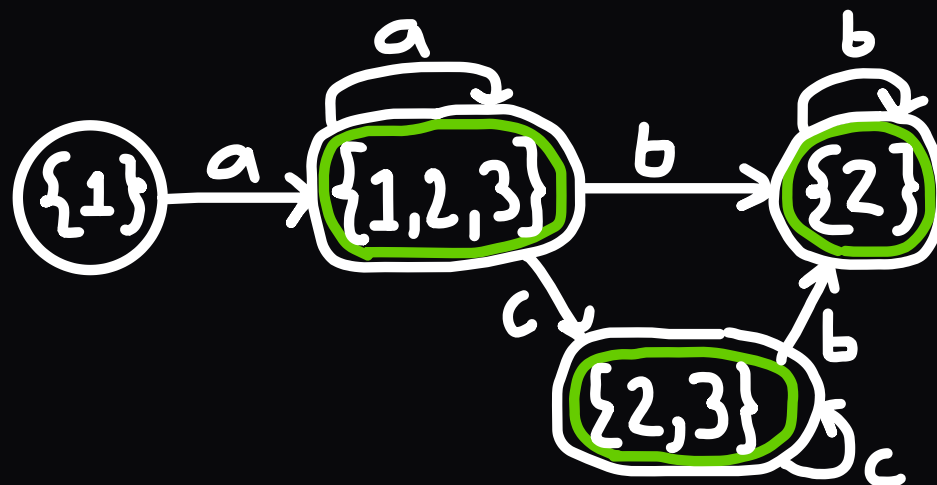


Ejemplo del profe I:

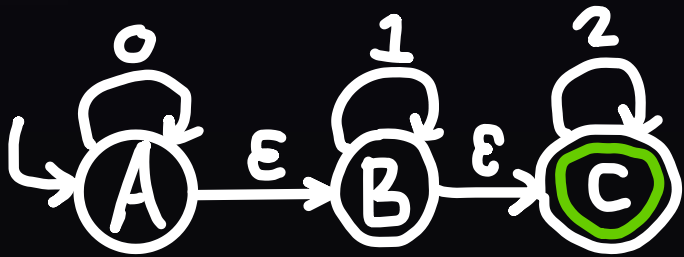


	a	b	c
{1}	{1, 2, 3}	$\emptyset$	$\emptyset$
{1, 2, 3}	{2}	{2, 3}	{2, 3}
{2}	$\emptyset$	{2}	$\emptyset$
{2, 3}	$\emptyset$	{2}	{2, 3}

Estado	Cierre- ϵ
1	{1}
2	{2}
3	{2, 3}

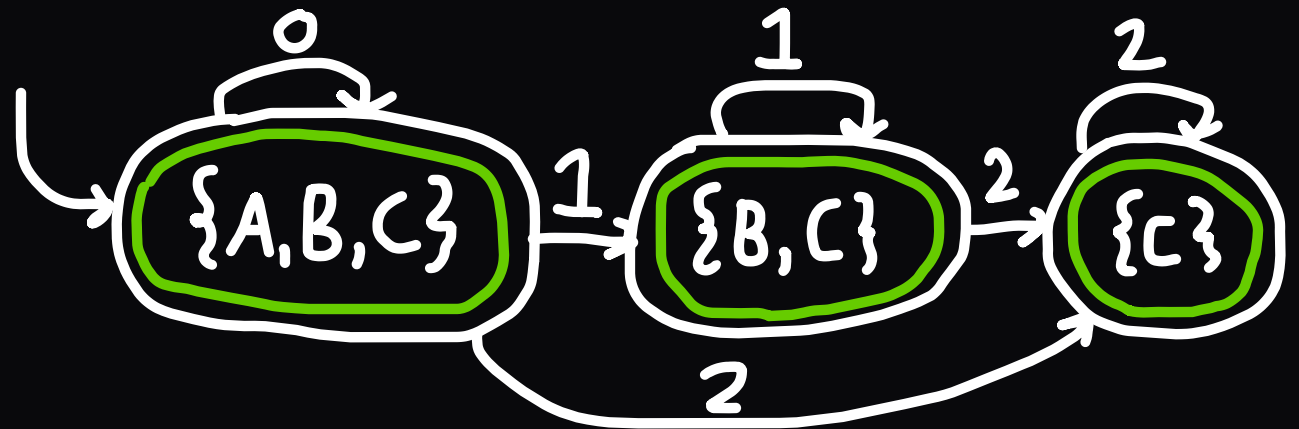


Ejemplo del profe II:

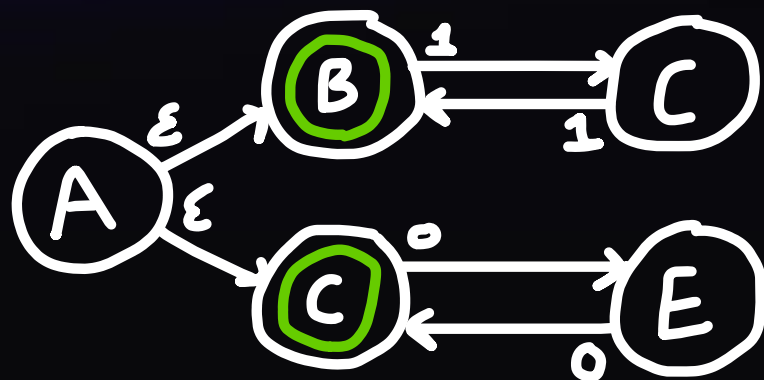


Estado	Cierre- ϵ
A	{A, B, C}
B	{B, C}
C	{C}

	0	1	2
{A, B, C}	{A, B, C}	{B, C}	{C}
{B, C}	$\emptyset$	{B, C}	{C}
{C}	$\emptyset$	$\emptyset$	{C}

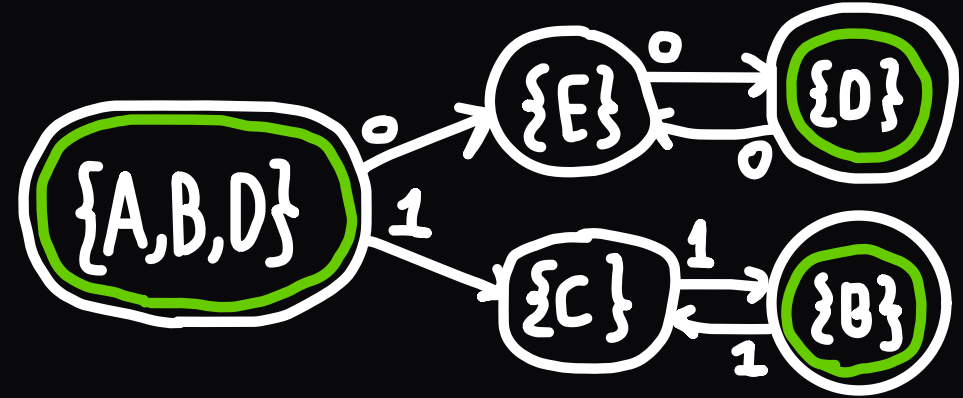


Ejemplo del profe III:



	0	1
{A, B, D}	{E}	{C}
{B}	{C}	$\emptyset$
{C}	{B}	$\emptyset$
{D}	$\emptyset$	{E}
{E}	$\emptyset$	{D}

Estado	Cierre- ϵ
A	{A, B, D}
B	{B}
C	{C}
D	{D}
E	{E}



Análisis léxico

Lenguajes Regulares

Un lenguaje $\mathcal{L}$ es regular $\Leftrightarrow \exists$ una DFA que lo reconozca.

Osea que acepta todas las hileras de $\mathcal{L}$ y rechaza cualquier hilera que no sea miembro de $\mathcal{L}$.

Recordemos el teorema que dice que las NFA y NFA- ϵ son “igual de poderosas” que las DFA, por lo que un lenguaje regular se puede definir así también:

Un lenguaje $\mathcal{L}$ es regular $\Leftrightarrow \exists$ una NFA que lo reconozca.
 $\Leftrightarrow \exists$ una NFA- ϵ que lo reconozca.

Análisis léxico

Lenguajes Regulares

Si queremos demostrar que un lenguaje no es regular hay que demostrar que no existe una DFA que lo reconozca, demostrar que algo es imposible es complicado D:

Para eso está el “Pumping Lemma”, una mauskerramienta misteriosa que nos va a ayudar más tarde. Todavía no estamos listos...

Todas las DFA, NFA y NFA- ϵ vistas en clase son, por definición, lenguajes regulares.

Recordemos que se podían hacer operaciones con lenguajes, ¿Cómo funciona eso con las DFA?

Unión $\cup$

Teorema: $\mathcal{L}, \mathcal{M}$ son lenguajes regulares $\Leftrightarrow \mathcal{L} \cup \mathcal{M}$ es un lenguaje regular.

Premisas:

- $\mathcal{L}$ es un lenguaje regular $\Leftrightarrow \exists$ una NFA- ϵ que reconozca $\mathcal{L}$.
- $\mathcal{M}$ es un lenguaje regular $\Leftrightarrow \exists$ una NFA- ϵ que reconozca $\mathcal{M}$.

Si logramos crear una DFA que reconozca $\mathcal{L} \cup \mathcal{M}$, entonces es regular.

Si pegamos las dos máquinas con una transición ϵ , ahora va a reconocer los dos (recordemos que una NFA- ϵ acepta cuando al menos uno de sus estados posibles es de aceptación).



Complemento

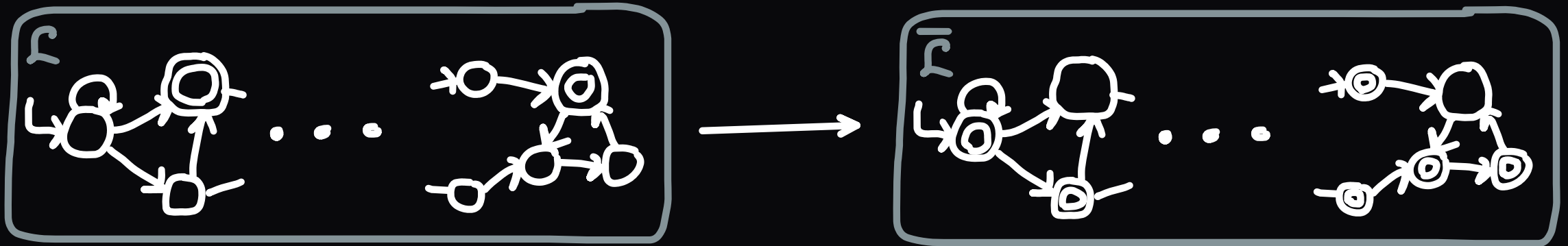
Teorema: L es un lenguaje regular $\Leftrightarrow \overline{L}$ es un lenguaje regular.

Premisas:

- L es un lenguaje regular $\Leftrightarrow \exists$ una NFA- ϵ que reconozca L .

Si logramos crear una DFA que reconozca $\overline{L}$, entonces es regular.

Hay que invertir los estados de aceptación (con cuidado de invertir los "pitfalls" también).



Intersección $\cap$

Teorema: $\mathcal{L}, \mathcal{M}$ son lenguajes regulares $\Leftrightarrow \mathcal{L} \cap \mathcal{M}$ es un lenguaje regular.

Premisas:

- $\mathcal{L}$ es un lenguaje regular $\Leftrightarrow \exists$ una NFA- ϵ que reconoce $\mathcal{L}$.
- $\mathcal{M}$ es un lenguaje regular $\Leftrightarrow \exists$ una NFA- ϵ que reconoce $\mathcal{M}$.

Si logramos crear una DFA que reconozca $\mathcal{L} \cap \mathcal{M}$, entonces es regular. Esto puede sonar complicado, pero recordemos que ya sabemos que unión e intersección son cerrados. Si usamos un poco de magia negra:

$$\mathcal{L} \cap \mathcal{M} = \overline{\overline{\mathcal{L} \cap \mathcal{M}}} = \overline{\overline{\mathcal{L}} \cup \overline{\mathcal{M}}}$$

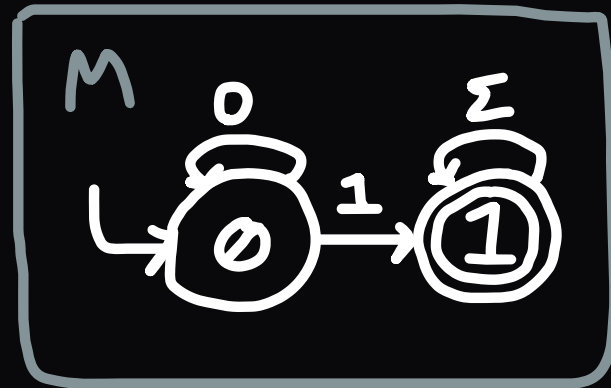
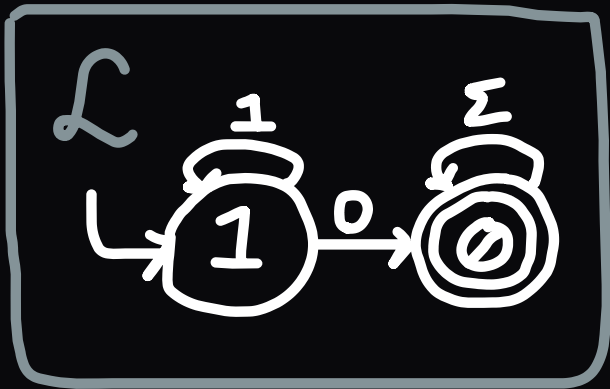
$\searrow$ De Morgan

Usando complementos e intersecciones (que ya sabemos hacer), obtenemos la intersección.

Ejemplo de $\cap$

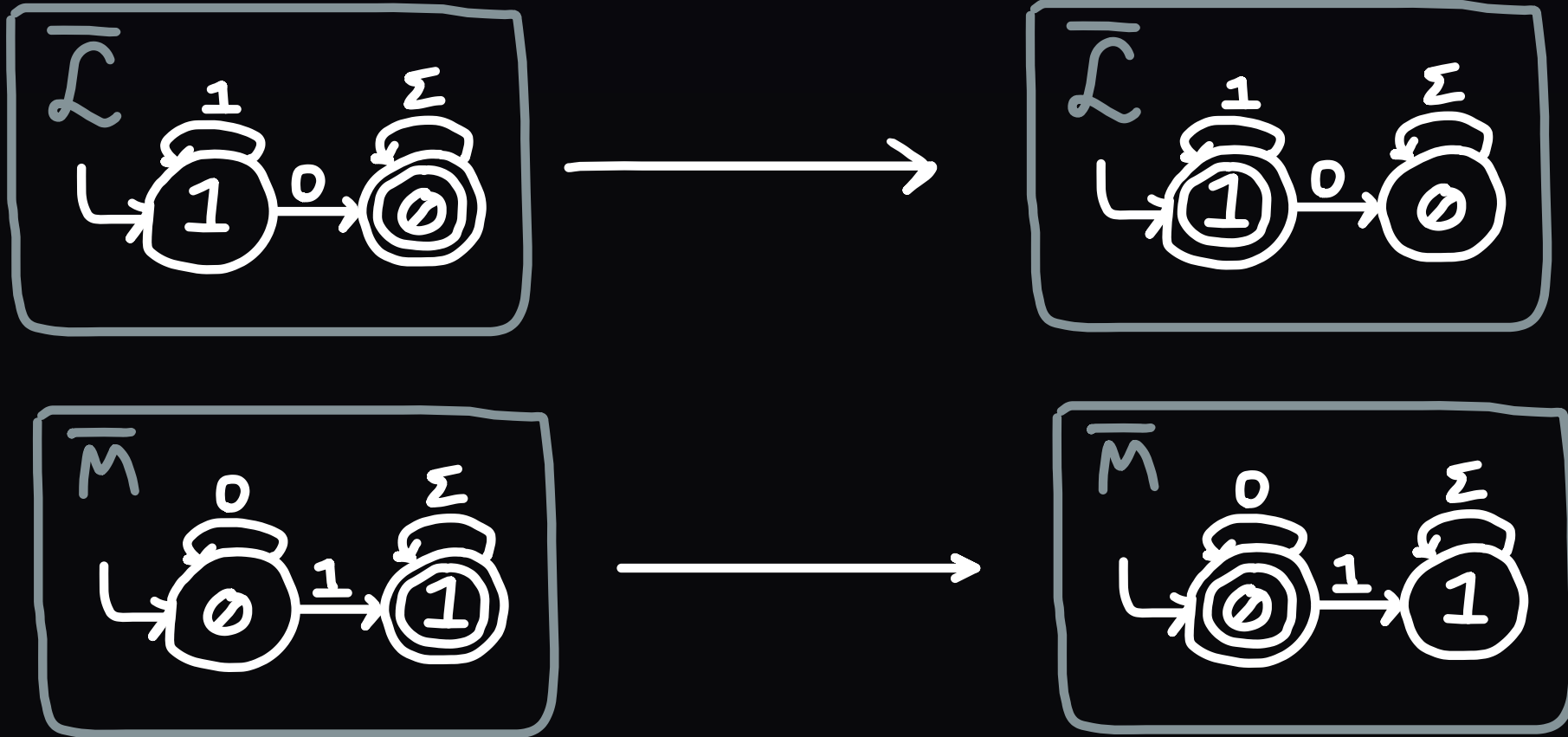
Exclusivamente porque me gusta sufrir voy a dar un ejemplo de intersección :D
Sean $\mathcal{L}$ y $\mathcal{M}$ dos lenguajes regulares sobre $\Sigma = \{0, 1\}$ tal que:

- $\mathcal{L}$ es el lenguaje sobre Σ de hileras que tengan al menos un 0.
- $\mathcal{M}$ es un lenguaje sobre Σ de hileras que tengan al menos un 1.



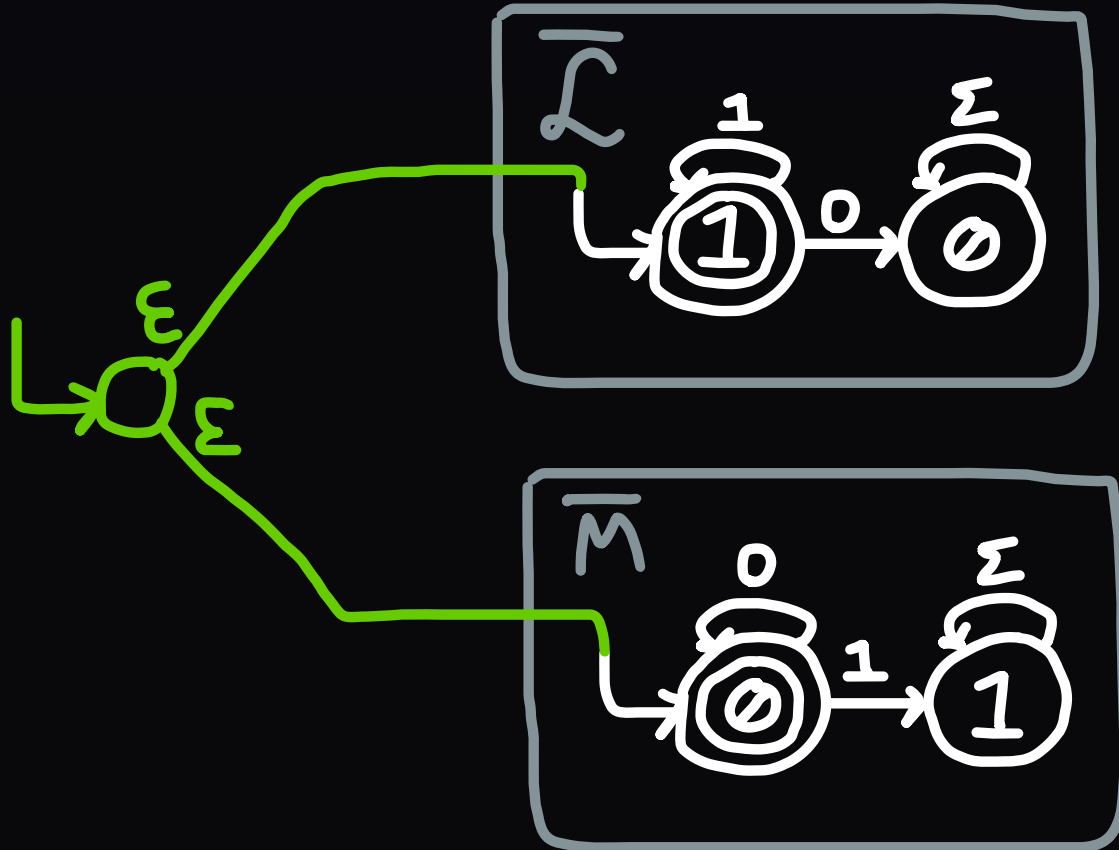
Ejemplo de n

Primero hay que calcular $\overline{L}$ y $\overline{M}$, se invierten los estados de aceptación



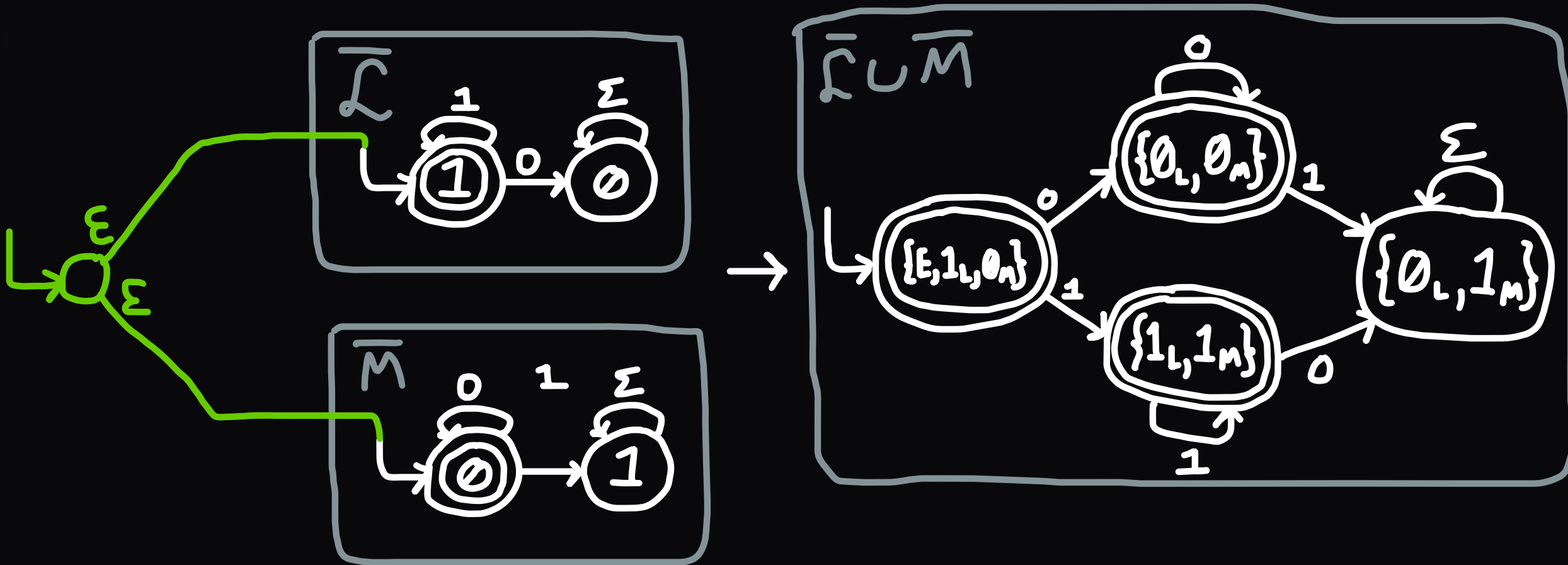
Ejemplo de n

Después se unen con una transición ε



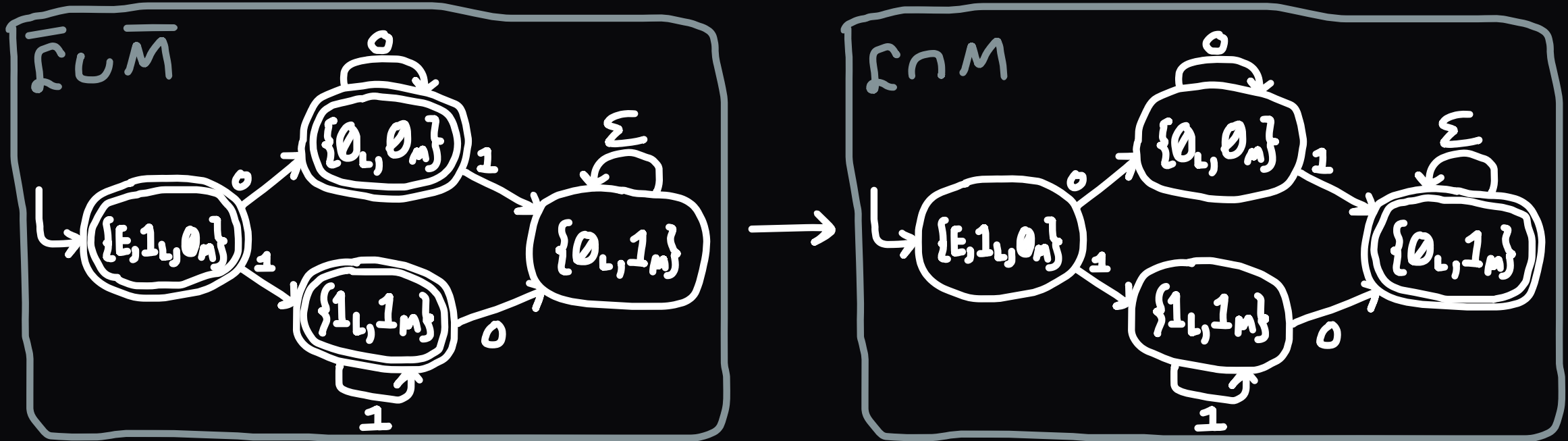
Ejemplo de n

Hay que convertir la NFA- ϵ en una DFA



Ejemplo de n

Por último, hay que sacar el complemento de la máquina. Ahora tenemos una máquina que contiene las hileras que tienen al menos un 0 y un 1.



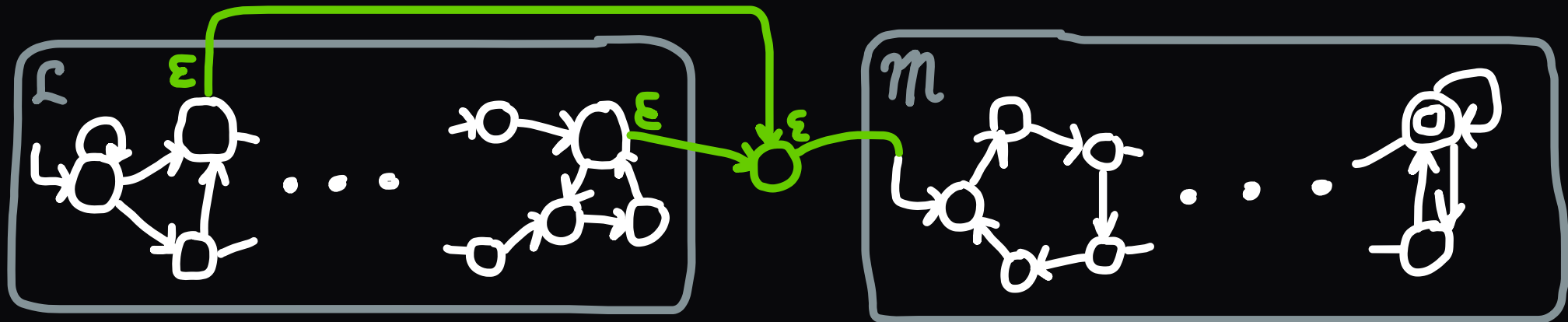
Concatenación

Teorema: $\mathcal{L}$, $\mathcal{M}$ son lenguajes regulares $\Leftrightarrow \mathcal{LM}$ es un lenguaje regular.

Premisas:

- $\mathcal{L}$ es un lenguaje regular $\Leftrightarrow \exists$ una NFA- ϵ que reconozce $\mathcal{L}$.
- $\mathcal{M}$ es un lenguaje regular $\Leftrightarrow \exists$ una NFA- ϵ que reconozce $\mathcal{M}$.

Si logramos crear una DFA que reconozca $\mathcal{LM}$, entonces es regular. Podemos agarrar todos los estados de aceptación de $\mathcal{L}$, pegarles transiciones ϵ , hacer que dejen de ser de aceptación, y poner transición ϵ en el estado inicial de $\mathcal{M}$.



¡Gracias por ver!
:D